

© Akshat Paul and Abhishek Nalwaya 2019

Akshat Paul and Abhishek Nalwaya, *React Native for Mobile Development*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-4454-8_2

2. The Simplest Program: Hello World with React Native

Akshat Paul¹ and Abhishek Nalwaya²

(1) Gurgaon, Haryana, India

(2) Jaipur, Rajasthan, India

Big things have small beginnings.

—Prometheus

In the last chapter, you got a good overview of the React ecosystem. Now it's time to get your hands dirty with React Native. In this chapter, you will set up your development environment by installing the prerequisites and then you will create your first React Native application.

The best way to learn is through practical examples. We continue this theme throughout the book, as you will follow simple examples to learn React Native by programming yourself to understand the key concepts.

This chapter explores the following topics:

- An introduction to React Native
- The essentials of React Native
- The installation of React Native
- Your first application
- The anatomy of a React Native application
- How to debug your application

Note You might face a situation where different projects work on different Node versions. Therefore, it's recommended you install Node Version Manager (NVM) to help keep multiple node versions that can be switched between projects.

What Is React Native?

React Native is an open source platform for developing native mobile applications; it was developed largely by a team at Facebook. The cool part of working with React Native is that your program uses standard web technologies like JavaScript (JSX), CSS, and HTML, yet your application is fully native. In other words, your application is fast and smooth, and it is equivalent to any native application built using traditional iOS technologies like Objective-C and Swift. However, React Native does not compromise in terms of performance and overall experience, like popular hybrid frameworks that use web technologies to build iOS apps.

React Native aims to bring the power of React, which was explained in Chapter **1**, to mobile development. In the words of the React team,

“Learn once, write anywhere.” Working with React and React Native, you will see how many of your components built for the Web using React can be easily ported to your React Native iOS apps with little or no modification. React Native introduces a highly functional approach to constructing UIs that is very different from the traditional iOS development approach.

Although React Native was built by Facebook developers, it’s an open source project. The code is available at

<https://github.com/facebook/react-native>.

Installation

Let’s do a quick, one-time setup of React Native. React Native is an assortment of JavaScript and Swift code, so you need tools that create, run, and debug your native application written in JavaScript. Let’s go one by one.

Installing Node and npm

`Node.js` is an open source platform built on Chrome’s JavaScript runtime; it offers a way to easily build fast, scalable programs. `Node.js` allows you to run JavaScript in terminal, and helps create modules. If you are using `nvm`, which is the suggested way of using node type:

```
> nvm install node
```

If you don't want to use `nvm`, you can also install `Node.js` by running the following command in terminal:

```
>brew install node.
```

Homebrew is the package manager for Mac. You can also download the Node installer from <https://nodejs.org> and install it manually if you are using another operating system. For Windows operating systems you can visit <https://nodejs.org> and install Node using a wizard.

`npm` is also installed along with `node`, which is a package manager for `Node.js`. If you're from the iOS world, it's similar to CocoaPods.

Check your `Node` installation by running the following command in terminal:

```
>> node -v  
v10.8.0  
>> npm -v  
6.2.0
```

Installing the React Native Package

To use React Native, starting your project from scratch using `create-reactive-app` by Facebook is the best way to start. It is an `npm` module:

```
npm install -g create-react-native-app
```


Updating React Native

Both React Native and iOS are fast-moving frameworks. It is recommended that you update them every time a new release is available. Upgrading React Native is simple. First install module `react-native-git-upgrade`:

```
$ npm install -g react-native-git-upgrade
```

Next, run the following command to start the process of upgrading to the latest version:

```
$ react-native-git-upgrade
```

For more information on React Native upgrades, you can refer to the Facebook official documentation at

https://facebook.github.io/react-native/docs/upgrading_.

Note You should only need to update the global installation of `create-react-native-app` very rarely, and ideally never.

Your First App

Now that you are all charged up about React Native and have your system set up, it's time to create your first application. To keep things simple, in the beginning just follow along. Sometimes you might feel disconnected by monotonously typing in the code, but following along is enough for now. Remember that mimicry is a powerful form of

learning; it's how we learned most of our skills, such as talking, reading, and writing, and it's how you will learn to program with React Native. As you proceed, this method will help you understand thoroughly why you authored certain pieces of code.

Throughout the book, you will create one application and take it from just Hello World to a full-blown, distribution-level application, except in a few places, where we need to digress to explore a concept independently. Before you set it up, then, let's talk about the problem you plan to solve. The app you will create during the course of this book plans to solve a few housing problems; it will be a very primitive version of any popular property search application. Let's call it HouseShare. It will have some rudimentary features like listings, creating an entry, geolocating a property, and a few more. As you move along, you will see how various React Native features fit with your application.

That's quite a lot, but in this chapter you just create the basic structure for your project using React Native and some Hello World code.

Creating a Basic Skeleton

Fire up your terminal and type in the following command:

```
create-react-native-app HouseShare
...
...
...
```

Success! Created HouseShare at
/Users/abhisheknaIwaya/Documents/book/HouseShareInside that directory,
you can run several commands:

```
yarn start
```

Starts the development server so you can open your React Native app in the Expo application on your phone.

```
yarn run ios  
(Mac only, requires Xcode)
```

Starts the development server and loads your app in an iOS simulator.

```
yarn run android  
(Requires Android build tools)
```

Starts the development server and loads your app on a connected Android device or emulator.

```
yarn test
```

Starts the test runner.

```
yarn run eject
```

Removes this tool and copies build dependencies, configuration files, and scripts into the app directory. If you do this, you can't go back!

We suggest that you begin by typing this:

```
cd HouseShare  
yarn start
```

Happy hacking!

So far we have used Expo a few times, so what is Expo? Expo is an open source tool chain that is built around React Native to help build iOS and Android apps. Expo is the fastest way to kickstart your React Native development. Because it comes out of the box with React Native, you don't need to perform any additional setup on your machine. The only extra thing you need to do is to install the Expo application from the Apple App Store for iOS and the Google Play Store for Android. Using this app, you will be able to test and interact with the application you are building during the development stages.

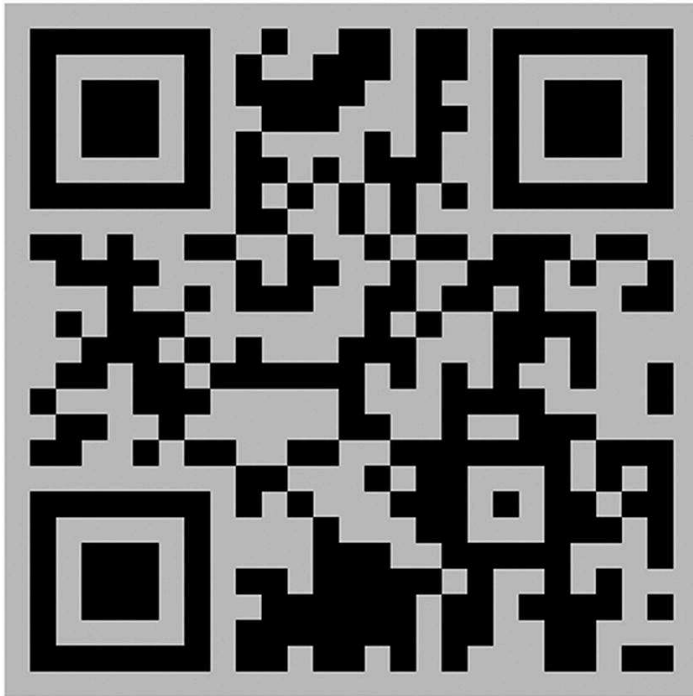
This code uses the CLI tool to construct a React Native project that is ready to build and run as is. This command creates the basic folder structure for your React Native iOS project.

```
> cd HouseShare  
> yarn start
```

You should see output similar to Figure [2-1](#).

This will start a development server for us and print a QR code in your terminal.

```
+ HouseShare yarn start
yarn run v1.3.2
$ react-native-scripts start
10:51:52 AM: Starting packager...
Packager started!
```



Your app is now running at URL: <exp://192.168.1.6:19000>

View your app with live reloading:

Android device:

-> Point the Expo app to the QR code above.
(You'll find the QR scanner on the Projects tab of the app.)

iOS device:

-> Press **s** to email/text the app URL to your phone.

Emulator:

-> Press **a** (Android) or **i** (iOS) to start an emulator.

Your phone will need to be on the same local network as this computer.
For links to install the Expo app, please visit <https://expo.io>.

Logs from serving your app will appear here. Press Ctrl+C at any time to stop.

- > Press **a** to open Android device or emulator, or **i** to open iOS emulator.
- > Press **s** to send the app URL to your phone number or email address
- > Press **q** to display QR code.
- > Press **r** to restart packager, or **R** to restart packager and clear cache.
- > Press **d** to toggle development mode. (current mode: **development**)

Figure 2-1 Terminal output when we build a React Native application

To use this QR code, download the Expo app (<https://expo.io/>) for iOS or Android on your device.

If you are using Android, just scan the QR code in your terminal from the Expo app and your app we automatically load. If you are using iOS, select “s” in your terminal, as shown in Figure [2-2](#).

Now open the e-mail, shown in Figure [2-3](#):

```
Please enter your phone number or email address (press ESC to cancel)
[[default: nalwayaabhishek@gmail.com]>
12:46:20 PM: Sending exp://192.168.1.6:19000 to nalwayaabhishek@gmail.com...
12:46:21 PM: Sent link successfully.

> Press a to open Android device or emulator, or i to open iOS emulator.
> Press s to send the app URL to your phone number or email address
> Press q to display QR code.
> Press r to restart packager, or R to restart packager and clear cache.
> Press d to toggle development mode. (current mode: development)

—
```

Figure 2-2 Press the s key if you are using iOS

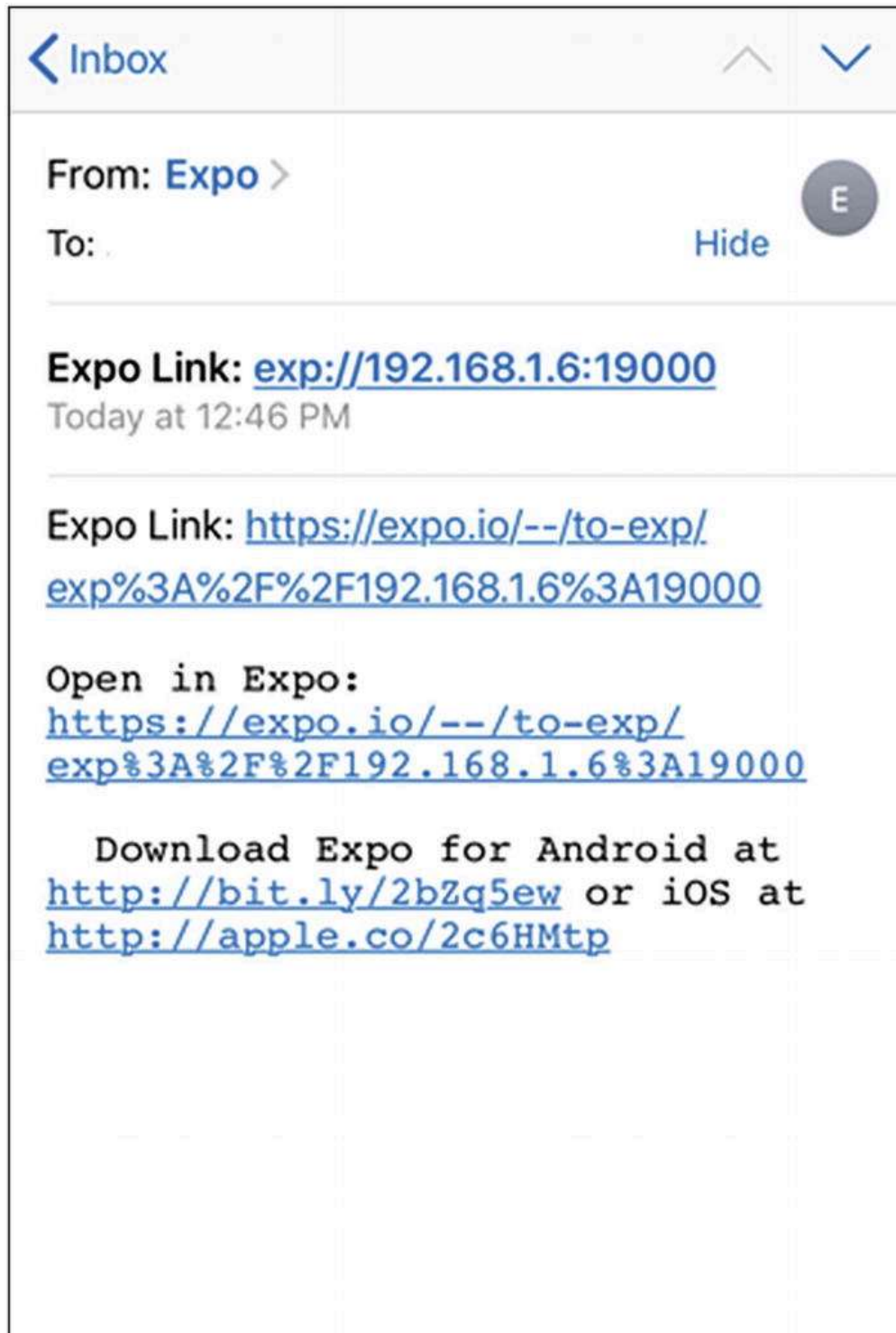




Figure 2-3 Expo link received in e-mail

Note Your mobile device needs to be connected to the same wireless network as your computer. Otherwise you will not be able to open the app.

If the Expo app is already installed on your device and you click the link it will automatically run the React Native app in the Expo app, as shown in Figure [2-4](#).

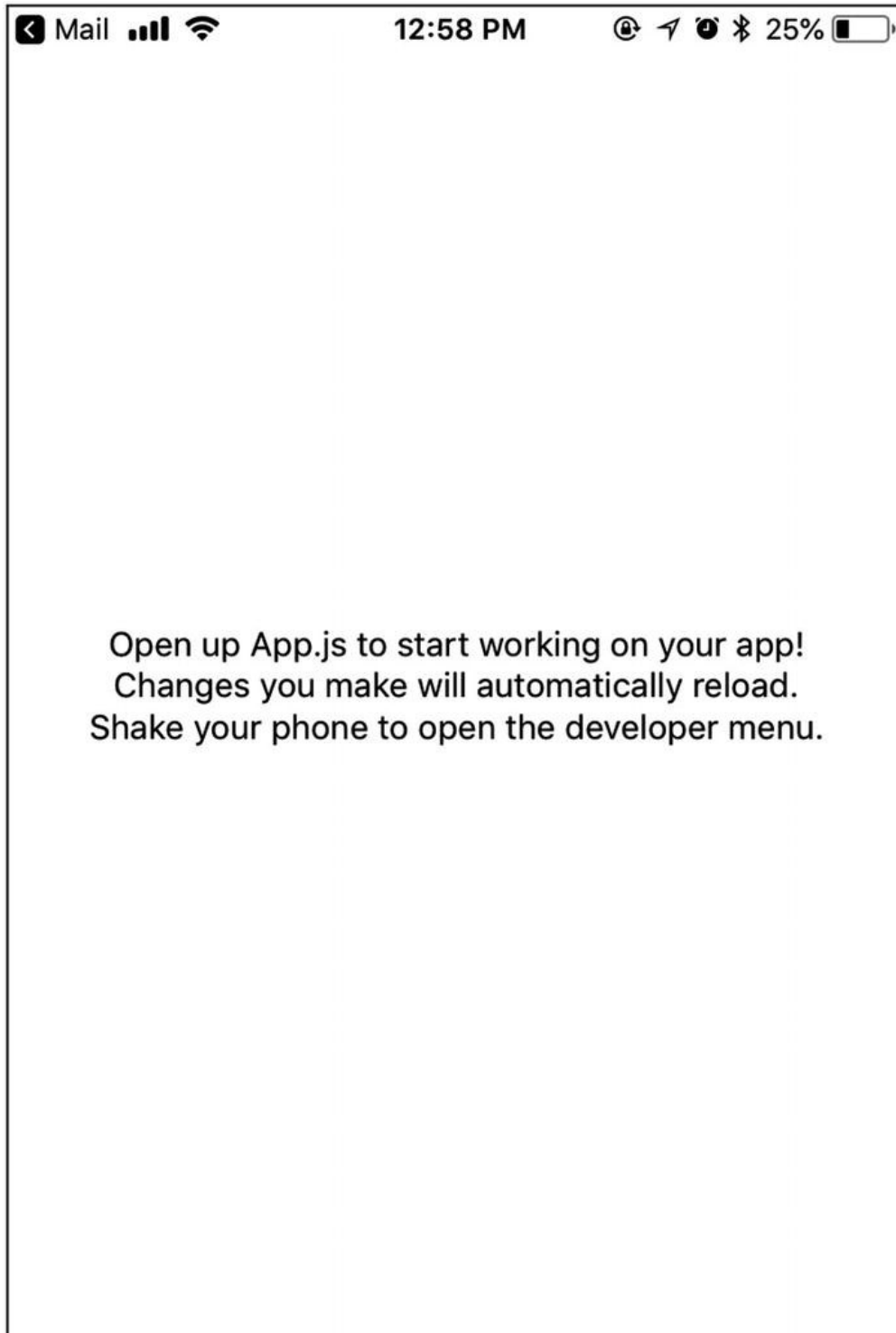




Figure 2-4 React Native application opened in Expo app

That was really quick and easy. Without installing the iOS and Android software development kit (SDK), we can run the app on our device using Expo.

Thanks to a single command, the basic structure of your project is in place and your application is loaded in the device. Also note that the terminal always needs to be open. This is the Node package manager for React Native. If you kill this, the app will stop working.

Terminal is opened to start the React Native Packager and a server to handle the preceding request. The React Native Packager is responsible for reading and building the JSX (you'll look at this later) and JavaScript code.

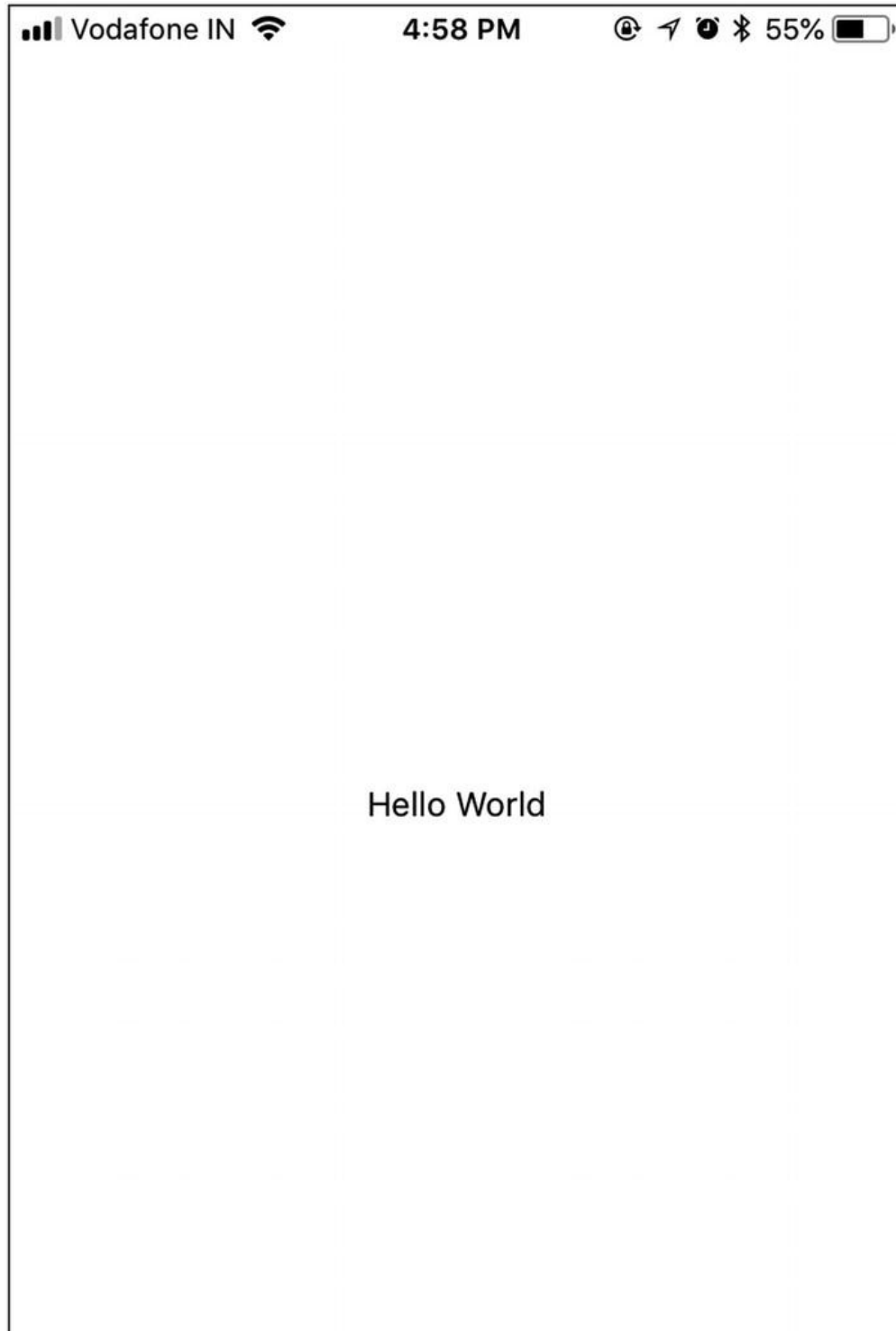
Set up your project in any editor you prefer. React Native does not force you to use nor does it have a preference for any specific editor, so you can continue to use your favorites.

Now let's update some code in our application. Add the following code in `App.js`:

```
import React from 'react';
import { StyleSheet, Text, View } from 'react-native';
export default class App extends React.Component {
  render() {
    return (
```

```
    <View style={styles.container}>
      <Text>
        Hello World
      </Text>
    </View>
  );
}
}
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

Just save the file, and then check the Expo app on your device. It automatically reloads the page and shows you the screen shown in Figure 2-5.



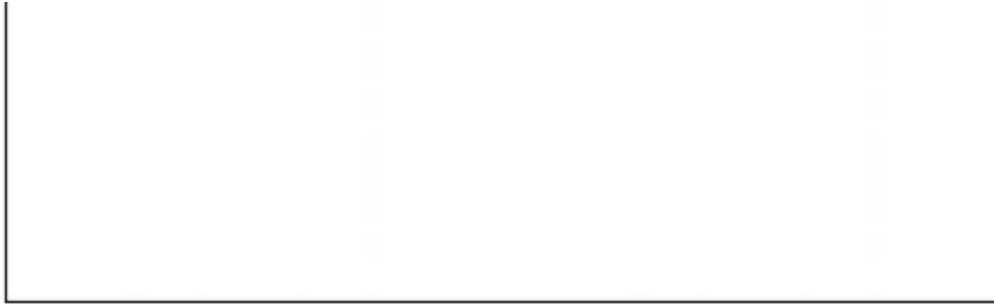


Figure 2-5 Updated text component appears on the screen on save

That was quick! In a fraction of a second you can see the changes you applied. You don't need to compile the code and restart the simulator for React Native changes. If you have done any native iOS app development before, pressing Refresh to see the changes might seem like a miracle.

Now, let's understand the code. At the top of the file are the following lines:

```
import React from 'react';  
import { StyleSheet, Text, View } from 'react-native';
```

This loads the `React` module and assigns it to a `React` variable that can be used in your code. React Native uses the same module-loading technology as `Node.js`; this is roughly equivalent to linking and importing libraries in Swift.

You are assigning multiple object properties to a single variable; this is called destructuring the assignment. This cool feature is in there in versions of JavaScript after ES6. Although it is optional, it's very beneficial; otherwise, every time you use a component in your code, you

would have to use a fully qualified name for it, such as `React.StyleSheet`, and so on. This saves quite a bit of time.

Next, you create a view:

```
export default class App extends React.Component {  
  render() {  
    return (  
      <View style={styles.container}>  
        <Text>  
          Hello World  
        </Text>  
      </View>  
    );  
  }  
}
```

React basic building blocks are called components. You can use the `React.Component` method to create custom component classes. This class has just one function, `render()`, which is responsible for what is shown on the screen. You use JavaScript syntax extensions (JSX) for rendering the UI. JSX is a JavaScript syntax extension that looks similar to XML.

Now you define the styling of your app. Here you will use Flexbox; it is similar to what CSS is to HTML. For now, you can type this code. We explain styling in the next chapter.

```
const styles = StyleSheet.create({
```

```
container: {  
  flex: 1,  
  backgroundColor: '#fff',  
  alignItems: 'center',  
  justifyContent: 'center',  
},  
});
```

You can see that this styling is very similar to CSS; you can define font size, alignment, and so on.

Prerequisites for Running App on a Simulator

Using the Expo iOS or Android application to test your app, there is a downside: You can't always carry your devices for testing your application. For such purpose there are simulators provided by both iOS and Android to be set up on your development machine. The following are few prerequisites to set them up.

iOS

- iOS apps can be developed only on an Apple Mac with OSX installed. You need OSX version 11 or above.
- You need Xcode 9 or above, which includes the iOS SDK and simulators. React Native only supports iOS7 or above. Xcode can be downloaded from the Apple App Store.

- It's helpful if you are enrolled in the Apple iOS Developer Program. If you're not in the iOS Developer Program, you won't be able to do the following:
 - Test applications on actual devices.
 - Access beta OS releases.
 - Test flight for beta testing.
 - Submit your app to the App Store.

Android

- React Native requires a recent version of the Java SE Development Kit (JDK).
- Download and install Android Studio. Choose a Custom setup when prompted to select an installation type. Make sure the check boxes next to all of the following are selected:
 - Android SDK
 - Android SDK Platform
 - Performance (Intel ® HAXM)
 - Android Virtual Device
- Install Android Virtual Devices (AVDs) by opening the AVD Manager from within Android Studio. You can also use genymotion.

Running the App on a Simulator

Now let's go back to our application and start the app (see Figure [2-6](#)):

```
yarn start
```

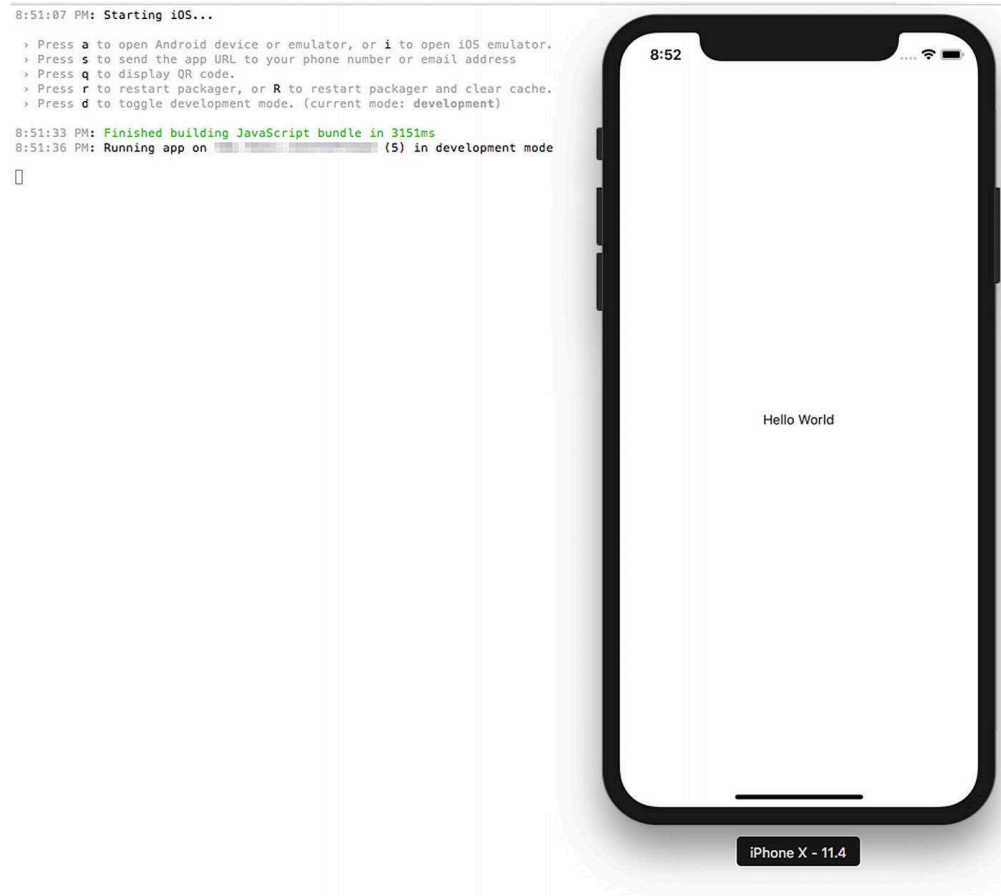


Figure 2-6 Running demo application on simulator

This will install Expo client on the emulator and run your React Native app. You can also use commands like `yarn ios` or `yarn android` to start the simulator with the app installed in it rather than loading the app inside the Expo simulator app.

It's Not a UIWebView

You are using web technologies, but your app does not have a web component; it has a native component. Open Debug ► View Debugging ► Capture View Hierarchy (see Figure [2-7](#)).

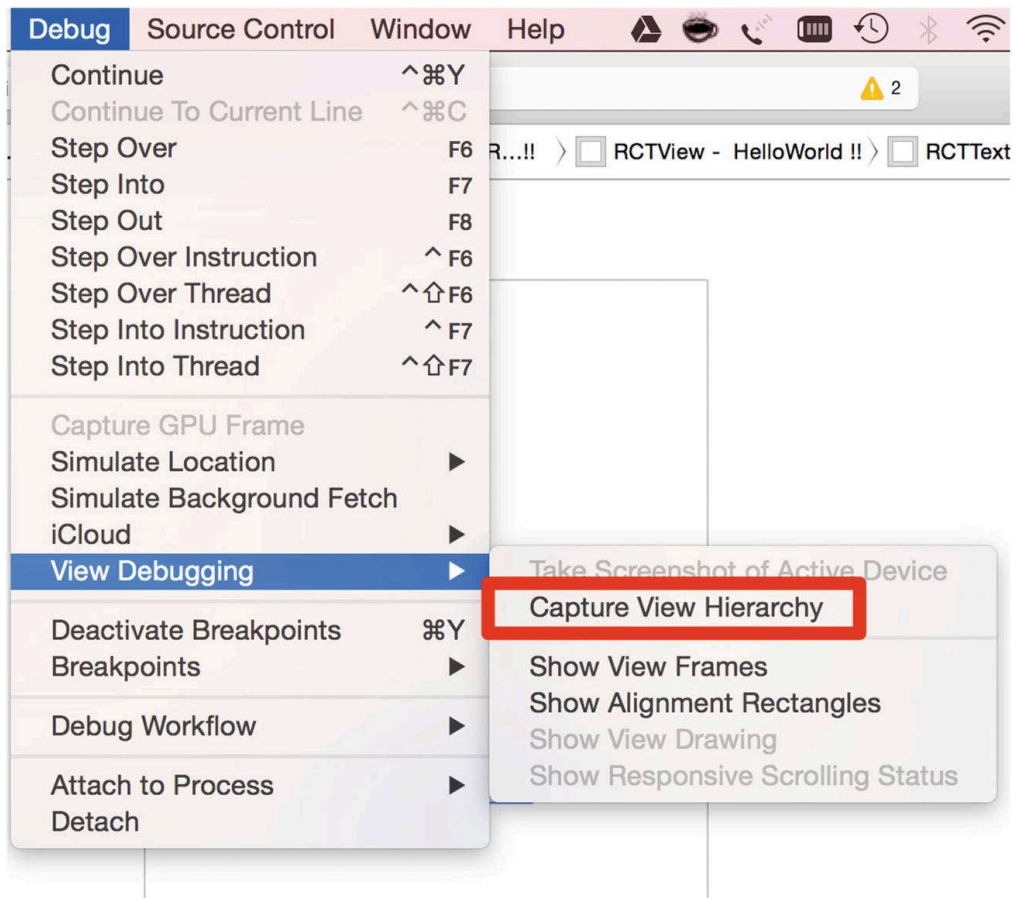


Figure 2-7 Using the Native component

As you traverse through the tree of `UIWindow`, you'll see that there is no `UIWebView` in the code, and "Hello World !!" is the call of `RCTText`, as shown in Figure 2-8.

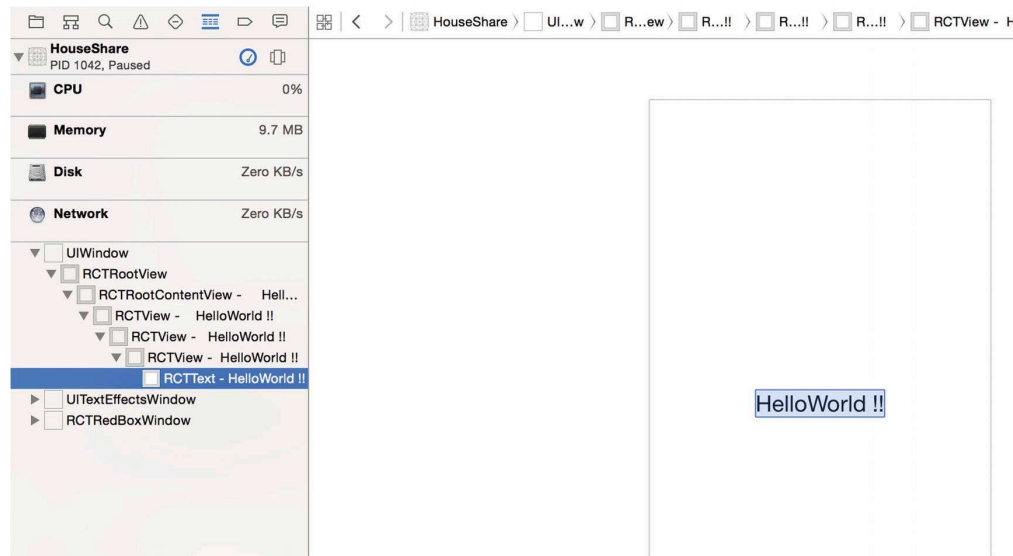


Figure 2-8 “Hello World !!” is the call of `RCTText`

Enabling Live Reload

Another cool feature of React Native is live reload. It reloads your application view inside the iOS simulator the moment there is a change. By default it is active. To deactivate this option, you need to access the developer menu from the application opened in the iOS simulator by shaking the device and then selecting the Disable Live Reload option.

What Makes React Native Different?

Before you dive deeper into the React Native world, you must understand why there was a need for another framework to build mobile apps. We already live in a world full of frameworks and tool chains that are capable of building mobile apps. Prior to the inception of React Native, building mobile apps using web technologies was possible via two strategies:

- *WebView-based*: These frameworks use common web technologies like HTML and JavaScript and use WebView to load the application. An example is the popular framework Phonegap.
- *Native apps using web technologies*: These frameworks again use common web technologies like HTML and JavaScript (to be precise, they imitate using JavaScript and HTML) to create native apps. An example is the popular framework Titanium Appcelerator.

Apps created using these strategies have performance issues.

WebView-based apps are slow because they use the DOM, and DOM manipulations are expensive, which leads to performance issues. As stated in a blog post at Flipboard (see

<http://engineering.flipboard.com/2015/02/mobile-web/>), “You cannot build a 60fps scrolling list view with DOM.” This is one of the fundamental problems with apps developed through this technique: Although development time might be quick, you end up with a sluggish experience.

The other strategy, where the framework imitates JavaScript and HTML, and converts them to native code, has other challenges.

Although the final app is native in nature, there is a basic issue during this conversion from JavaScript to native: It runs on the main thread. In these apps, you interface directly with native objects all the time, which leads once again to a slow and sluggish experience.

React Native is fundamentally different from these two approaches. It runs all layouts on separate threads, and your main thread is free to update the UI, which makes the animation and UI rendering smooth, just like 100 percent pure native apps.

React Native uses the JavaScriptCore framework to run JavaScript. In iOS 7, Apple introduced a native Objective-C API for JavaScriptCore. This framework allows JavaScript and Objective-C to talk to each other. This means you can create and call JavaScript functions from Objective-C or call back into Objective-C from JavaScript. It all works like a charm.

React Native is different in one more aspect. As seen in your Hello World example, you write a component in JavaScript just like you would with React, except that instead of using an HTML `div`, you use tags like `View` and `Text`. In the case of an iOS application, a `View` is basically a `UIView`.

Ejecting a React Native Application

Before we get into exploring the application structure, we have to eject our application from the Expo project (see Figure [2-9](#)). You should not eject an app from the Expo environment until it is needed. We are doing this here to understand how `create-react-native` works.

```
yarn eject
```

```

[+] HouseShare yarn eject
yarn run v1.3.2
$ react-native-scripts eject

We didn't find any uses of the Expo SDK in your project, so you should be fine to eject to
"Plain" React Native. (This check isn't very sophisticated, though.)

We strongly recommend that you read this document before you proceed:
https://github.com/react-community/create-react-native-app/blob/master/EJECTING.md

Ejecting is permanent! Please be careful with your selection.

? How would you like to eject from create-react-native-app? React Native: I'd like a regular React Native project.
We have a couple of questions to ask you about how you'd like to name your app:
[?] What should your app appear as on a user's home screen? HouseShare
[?] What should your Android Studio and Xcode projects be called? HouseShare
Writing your selections to app.json...
Wrote to app.json, please update it manually in the future.
Scanning folders for symlinks in /Users/abhisheknalwaya/Documents/book/HouseShare/node_modules (21ms)
Generating the iOS folder.
Generating the Android folder.
Successfully copied template native code.
Babel preset changed to 'babel-preset-react-native-stage-0/decorator-support'.
Updating your yarn scripts in package.json...
Your package.json is up to date!
Adding entry point...
Added new entry points!

Note that using 'yarn start' will now require you to run Xcode and/or
Android Studio to build the native code for your project.
Removing node_modules...
Installing packages with yarn...
[1/4] 🔍 Resolving packages...
[2/4] 📦 Fetching packages...
[3/4] 🔗 Linking dependencies...
warning "react-native > eslint-plugin-react-native@3.2.1" has unmet peer dependency "eslint@^3.17.0 || ^4.0.0".
[4/4] 🏗 Building fresh packages...
success Saved lockfile.
success Saved 48 new dependencies.
├─ babel-jest@23.4.2
├─ babel-plugin-istanbul@4.1.6
├─ babel-plugin-jest-hoist@23.2.0
├─ babel-plugin-syntax-class-constructor-call@6.18.0
├─ babel-plugin-syntax-do-expressions@6.13.0
├─ babel-plugin-syntax-function-bind@6.13.0
├─ babel-plugin-transform-class-constructor-call@6.24.1
├─ babel-plugin-transform-do-expressions@6.22.0
├─ babel-plugin-transform-export-extensions@6.22.0
├─ babel-plugin-transform-function-bind@6.22.0
├─ babel-preset-jest@23.2.0
├─ babel-preset-react-native-stage-0@1.0.1
├─ babel-preset-react-native@1.9.2
├─ browser-resolve@1.11.3
├─ expect@23.5.0
├─ istanbul-api@1.3.1
├─ jest-changed-files@23.4.2
├─ jest-cli@23.5.0
├─ jest-config@23.5.0
├─ jest-diff@23.5.0
├─ jest-docblock@23.2.0
├─ jest-each@23.5.0
├─ jest-environment-jsdom@23.4.0
├─ jest-environment-node@23.4.0
├─ jest-haste-map@23.5.0
├─ jest-jasmine2@23.5.0
└─ jest-leak-detector@23.5.0

```

Figure 2-9 Ejecting Expo application

This will create two folders for iOS and Android in the repository. If you open the project HouseShare, it looks like a normal Xcode project. It has the following folder structure:

```

| ios
|- HouseShare

```

```
| - HouseShare.xcodeproj
| - HouseShareTests
| android
node_modules
App.js
App.test.js
index.js
package.json
yarn.lock
```

Note The folder structure defined here might be changed or modified as the framework evolves, but the majority of the functionality remains the same.

If you open the project in Xcode, it will have a different folder structure. The “folders” in Xcode are actually groups and are not necessarily linked to a folder like we see in Finder.

- **iOS:** The `iOS` folder has two folders and one file. As seen earlier, there is a `HouseShare` folder, which has all the Objective-C code, such as `AppDelegate`, `Images.xcassets`, `Info.plist`, `LaunchScreen.xib`, and other files. Another folder is `HouseShareTests`, which is where all your test cases reside. Finally, there is your Xcode project file, `HouseShare.xcodeproj`, which is used to load into Xcode to build your application.
- **package.json:** This folder contains metadata about your app, and it will install all dependencies when you run the `npm install`. If you’re familiar with Ruby, it’s similar to a `Gemfile`.
- **node_modules:** All of the Node modules mentioned in `package.json` will be downloaded to this folder. This folder also

contains the code for the React Native framework.

- `App.js`: This is the file where you begin programming your application.
- `AppDelegate.m`: This is the starting point of any iOS app.
- `Android`: React Native also supports development for Android. All your native Android code resides in this folder.

Let's open the `AppDelegate.m` file from

`HouseShare/ios/HouseShare/AppDelegate.m`:

```
#import "AppDelegate.h"
#import <React/RCTBundleURLProvider.h>
#import <React/RCTRootView.h>
@implementation AppDelegate
- (BOOL)application:(UIApplication *)application didFinishLaunchingWithOptions:(NSDictionary *)launchOptions
{
    NSURL *jsCodeLocation;
    jsCodeLocation = [[RCTBundleURLProvider sharedSettings] jsBundleURLForBundleRoot:@"index" fallbackResource:nil];
    RCTRootView *rootView = [[RCTRootView alloc] initWithBundleURL:jsCodeLocation
        moduleName:@"HouseShare"
        initialProperties:nil
        launchOptions:launchOptions];
    rootView.backgroundColor = [[UIColor alloc] initWithRed:1.0f green:1.0f blue:1.0f alpha:1];
    self.window = [[UIWindow alloc] initWithFrame:[UIScreen mainScreen].bounds];
    UIViewController *rootViewController = [UIViewController new];
    rootViewController.view = rootView;
    self.window.rootViewController = rootViewController;
    [self.window makeKeyAndVisible];
    return YES;
}
@end
```

`RCTRootView` is a Swift class provided by React Native, which is inherited from the iOS `UIView` Class. It takes your JavaScript code and executes it. It also loads the `index bundle` URL, which has your code written in `App.js` and also a program added by the React Native framework.

Note After ejection, you need to use Xcode to run the iOS app and Android Studio to run the Android app.

To start, run `yarn start` on terminal, as shown in Figure 2-10.

```
[+ HouseShare yarn start
yarn run v1.3.2
$ react-native start
Scanning folders for symlinks in /Users/abhisheknalwaya/Documents/book/HouseShare/node_modules (23ms)

Running Metro Bundler on port 8081.

Keep Metro running while developing on any JS projects. Feel free to
close this tab and run your own Metro instance if you prefer.

https://github.com/facebook/react-native

Looking for JS files in
/Users/abhisheknalwaya/Documents/book/HouseShare

Metro Bundler ready.
Loading dependency graph, done.
```

Figure 2-10 Starting the application without Expo

Now open `HouseShare.xcodeproj`. This Xcode project file will open your project in Xcode. Next, let's load your application in the iOS simulator. To build your application and load it in the simulator, simply click the Run button at the top left (or execute `Command + R`), as shown in Figure 2-11. This will compile, build, and fire up your project in the iOS simulator

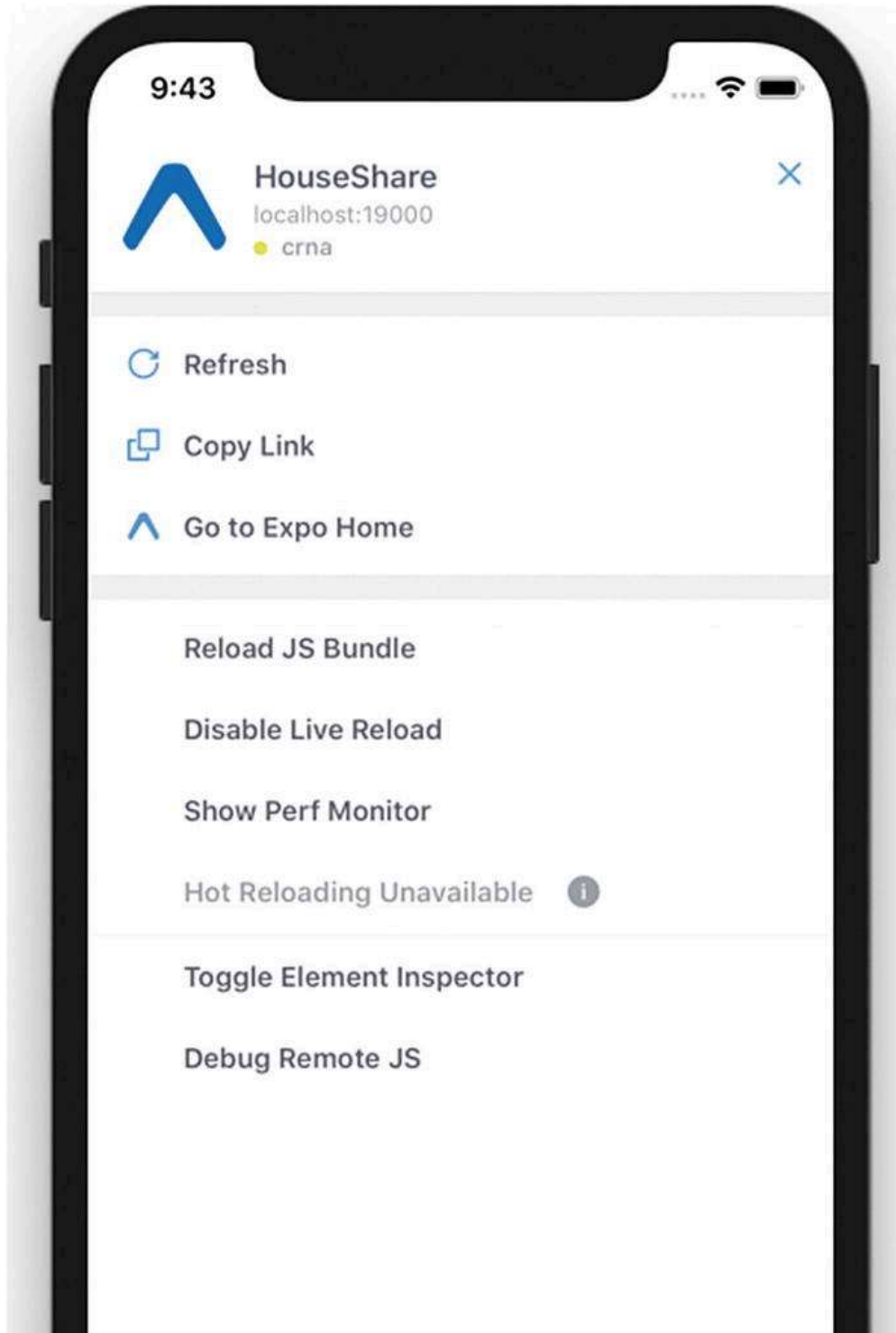


Figure 2-11 Building the application using Xcode

This will open the simulator and you can see the app running.

Debugging

Debugging with React Native is in line with how we debug web apps; in short, it's really simple. To access debugging options, share the simulator by selecting Share Gesture from the Hardware menu. This will open a menu that provides several debugging options, as shown in [Figure 2-12](#).



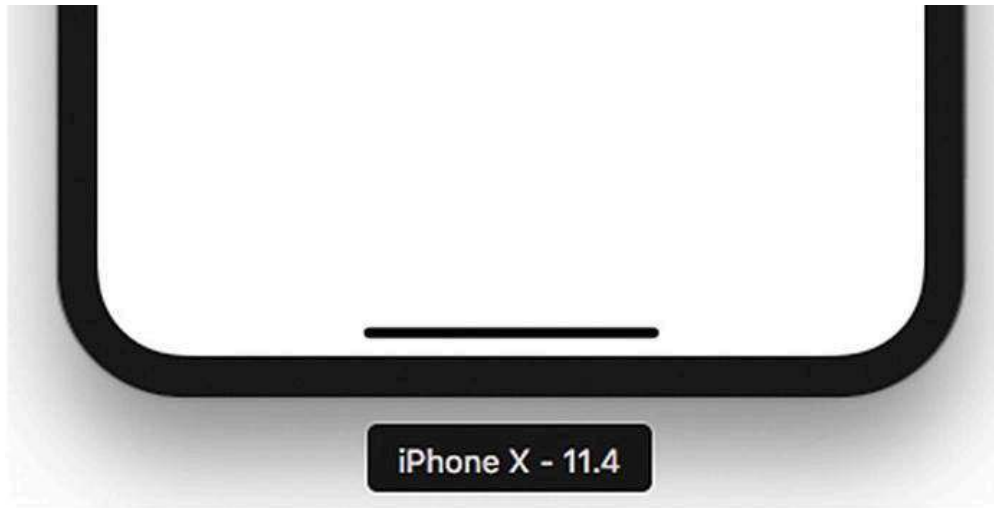


Figure 2-12 Debugging options for React Native applications

You must disable this menu for the final build because your end user should not see these options. To disable it, open the project in Xcode and select Product ► Scheme ► Edit Scheme (or press Command + <). Then select Run from the menu on the left and change the Build Configuration to Release.

Let's review each of the options shown in Figure [2-12](#).

Reload

The Reload option refreshes the screen in the simulator with the latest React Native code without compiling the project again. This can be done in two ways: selecting the Reload option from the menu or pressing Command + R. This will reload all the changes made in the JavaScript code.

Any changes made in your Swift or Objective-C files will not be reflected because these changes require recompilation. Also, if you add any assets like images, the app needs to be restarted.

Debugging in Chrome

This is one of the best and most frequently used options for debugging your JavaScript code written in React Native. As with web apps, you can debug your React Native application in Chrome. When you click Debug in Chrome, it opens `http://localhost:8081/debugger-ui` in Chrome (Figure 2-13).

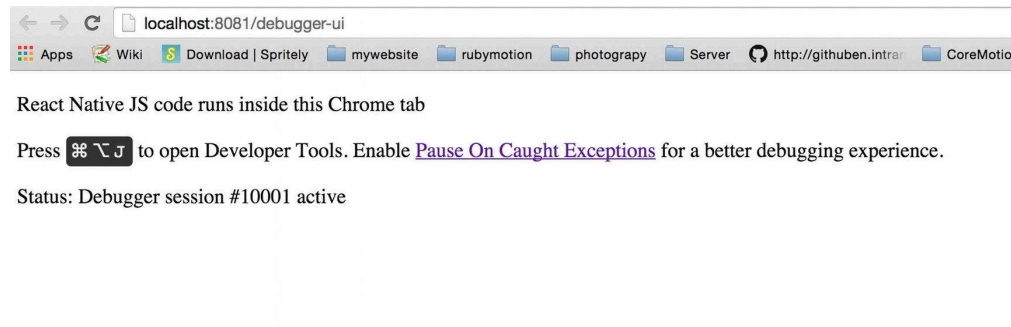


Figure 2-13 Debugging in Chrome

Install the React Developer Tools, which is a Chrome extension for debugging both your React application and React Native code. It allows you to inspect the React Native component hierarchies in the Chrome Developer Tools. To install it, please visit the Chrome webstore or go to <https://chrome.google.com/webstore/detail/react-developer-tools/fmkadmapgofadopljbjfkapdkoienihi?hl=en>.

Once the extension is installed, press Command + Option + J or select View ► Developer ► Developer Tools in your Chrome browser to ac-

cess the Developer Tools console.

You will see a new tab called React in your Chrome DevTools. This shows you the root React components that have been rendered on the page, as well as the subcomponents that they ended up rendering. You can also see props, state, components, and event listeners, as shown in Figure 2-14.

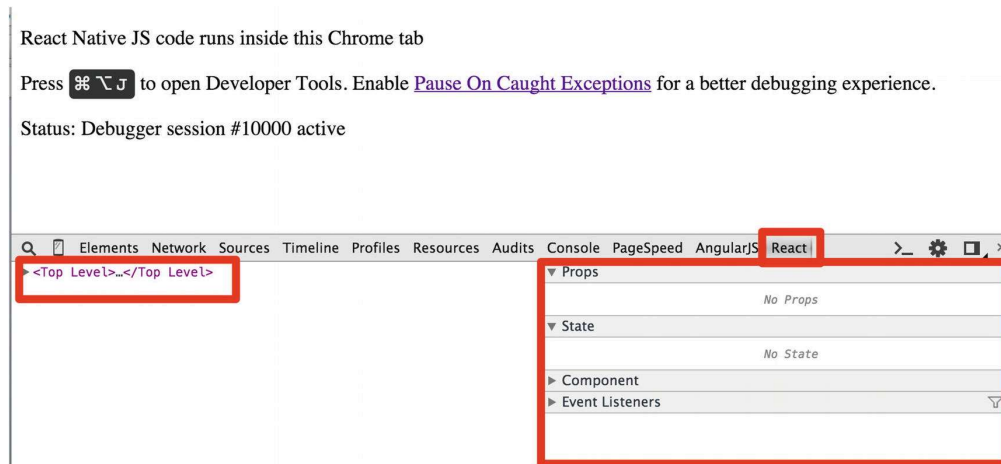


Figure 2-14 Debugging in Chrome DevTools

Look at Figure 2-15 and you can see a similar hierarchy to your Xcode: Hello World is wrapped in `RCTText` and that is in turn wrapped in `RCTView`.



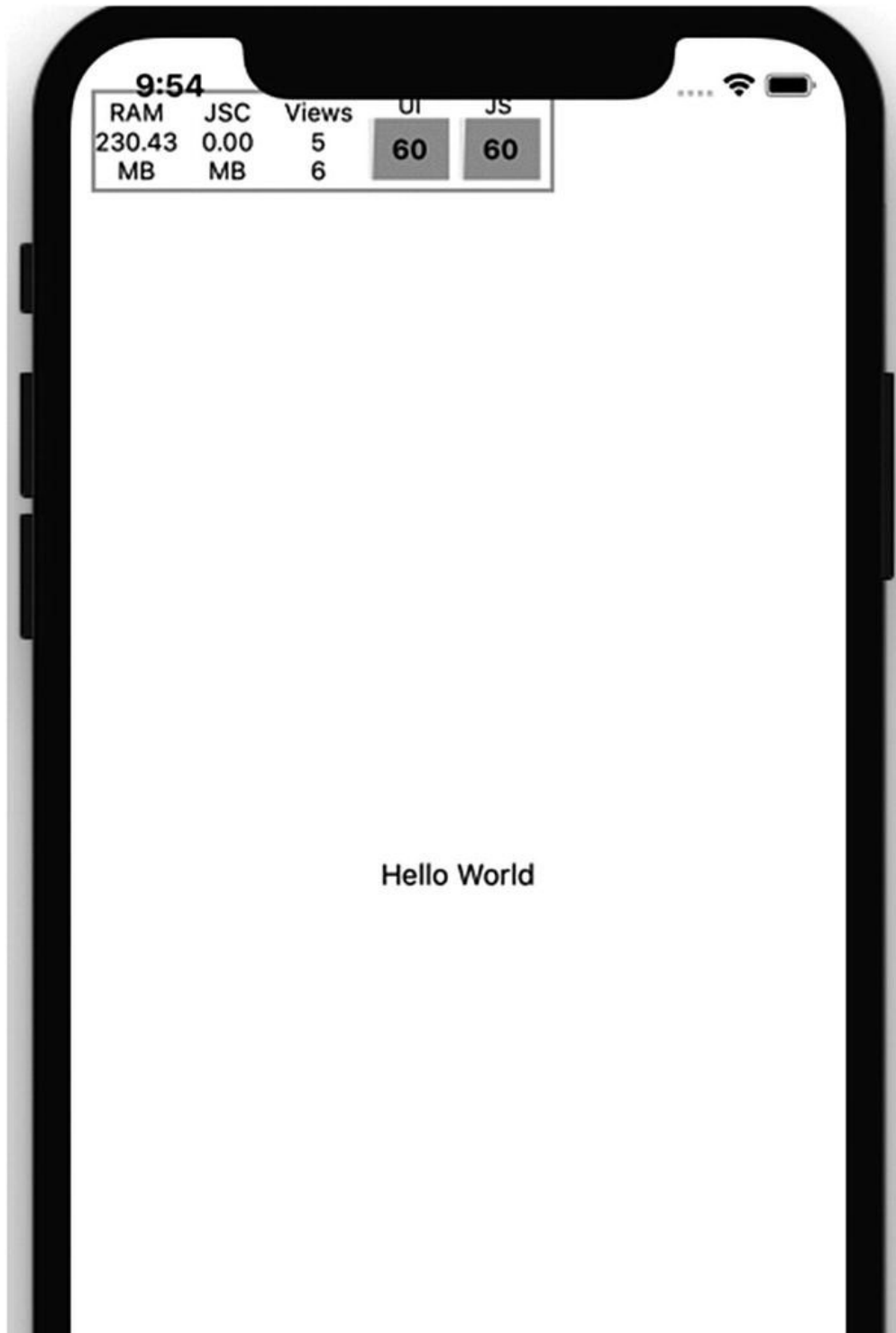
Figure 2-15 Debugging the app with the React tab in Chrome DevTools

Debugging in Safari

If you do not have Chrome, you can also use Safari for debugging, but Chrome is preferred for debugging React Native apps.

Showing Performance Monitor

Many applications use a lot of animations and graphics. The smoothness of these animations for your application is defined in frames per second (FPS); this is used extensively in gaming apps. When you select Show FPS Monitor from the menu, it shows a few properties for your app in the simulator (see Figure [2-16](#)). Although you might not find much use for these properties in your Hello World app, they are great for animation-intensive apps to prevent them lethargic performance, which can create a bumpy user experience.



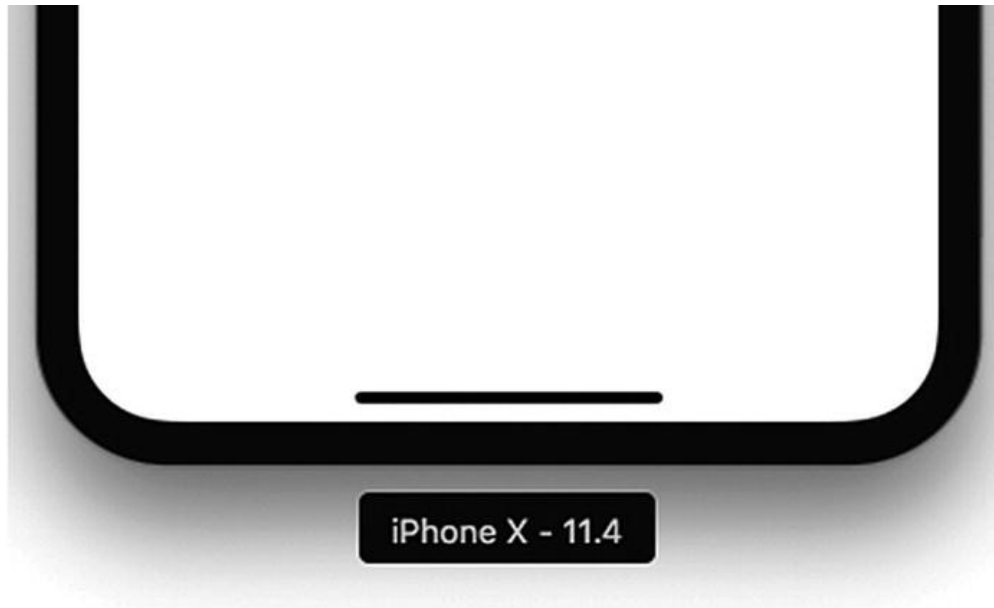


Figure 2-16 Additional properties in the simulator

The Inspect Element

You can also inspect a React Native element from the simulator, somewhat similar to how you inspect an element in a browser, although you can't currently change live values of properties as you can in a browser. For now, you can see your stylesheet properties for any object. Click the HelloReact!! text (Figure [2-17](#)) and it will open the details of that element.

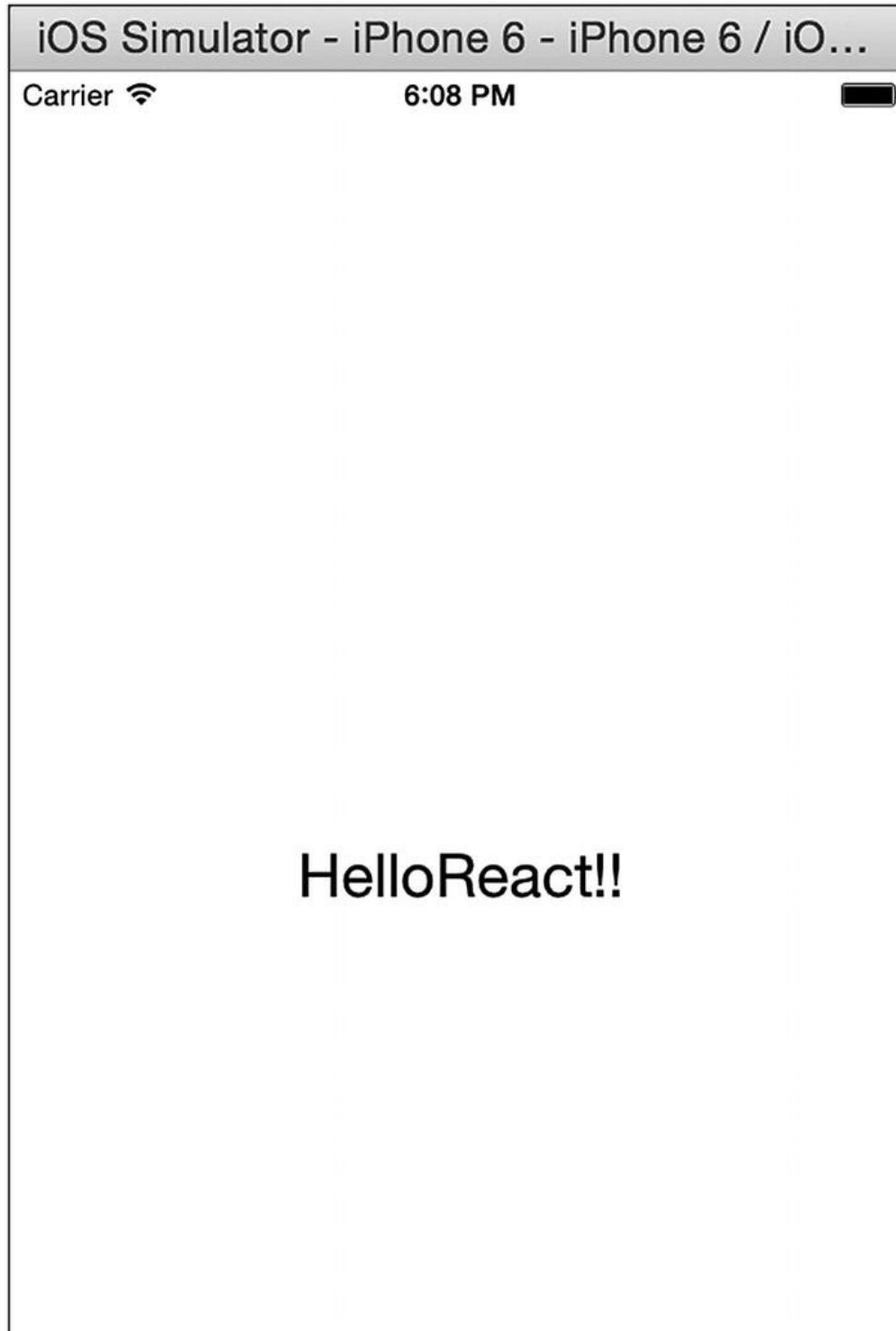


Figure 2-17 Click the text to see element details

The details of that element are shown in Figure [2-18](#) at the bottom left.

Welcome to the inspector! Tap something to inspect it.

